MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH & STUDIES
School of Computer Applications (SCA)

LAB PRACTICAL RECORD FILE

DATA STRUCTURES & ALGORITHMS

SUBMITTED TO:
Mr. Sachin Sharma
Faculty / Assistant Professor
School of Computer Applications
MRIIRS

SUBMITTED BY:
Dwadasi Venkata Sri Sai Kumar
Roll No: 26MCAP101N007
Course: MCA
Semester: 1

Academic Session 2026 - 2027

INDEX OF EXPERIMENTS

Exp. No.	Date	Title of Experiment	Page No.	Sign & Remarks
Exp 1	15-10-2026	List Operations (Insertion, Deletion and Display)	3-7	
Exp 2	15-10-2026	Sorting Algorithms & Time Complexity Comparison	8-11	
Exp 3	15-10-2026	Linear Search vs Binary Search & Time Comparison	12-14	
Exp 4	15-10-2026	Stack, Queue, Circular Queue and Browser History	15-23	
Exp 5	15-10-2026	Evaluation of Postfix Expression	24-25	
Exp 6	15-10-2026	Conversion of Infix to Postfix	26-27	
Exp 7	15-10-2026	Singly Linked List (Insert, Delete, Display)	28-33	
Exp 8	15-10-2026	Circular Linked List (Insert, Delete, Display)	34-39	
Exp 9	15-10-2026	Stack, Queue & Circular Queue using Linked List	40-47	
Exp 10	15-10-2026	Linked List Concatenation & Polynomial Addition	48-52	

EXPERIMENT 1: LIST OPERATIONS (INSERTION, DELETION AND DISPLAY)

AIM: To write and execute a Python program to perform basic operations on a list: inserting an element at a specified position, deleting an element by position, and displaying the list contents.

Python Source Code

```
# Simple program for list operations: Insertion, Deletion, Display

my_list = []

def insert():
    value = input("Enter value to insert: ")
    pos = input("Enter position (press Enter to add at end): ")
    if pos == "":
        my_list.append(value)
    else:
        my_list.insert(int(pos), value)
    print("Inserted successfully.")

def delete():
    if not my_list:
        print("List is empty. Nothing to delete.")
        return
    value = input("Enter value to delete: ")
    if value in my_list:
        my_list.remove(value)
        print("Deleted successfully.")
    else:
        print("Value not found in the list.")

def display():
    if not my_list:
        print("List is empty.")
    else:
        print("List elements:", my_list)

while True:
    print("\n--- LIST OPERATIONS ---")
    print("1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        insert()
    elif choice == "2":
        delete()
    elif choice == "3":
        display()
    elif choice == "4":
        print("Program ended.")
        break
    else:
        print("Invalid choice. Try again.")
```

Space for Output

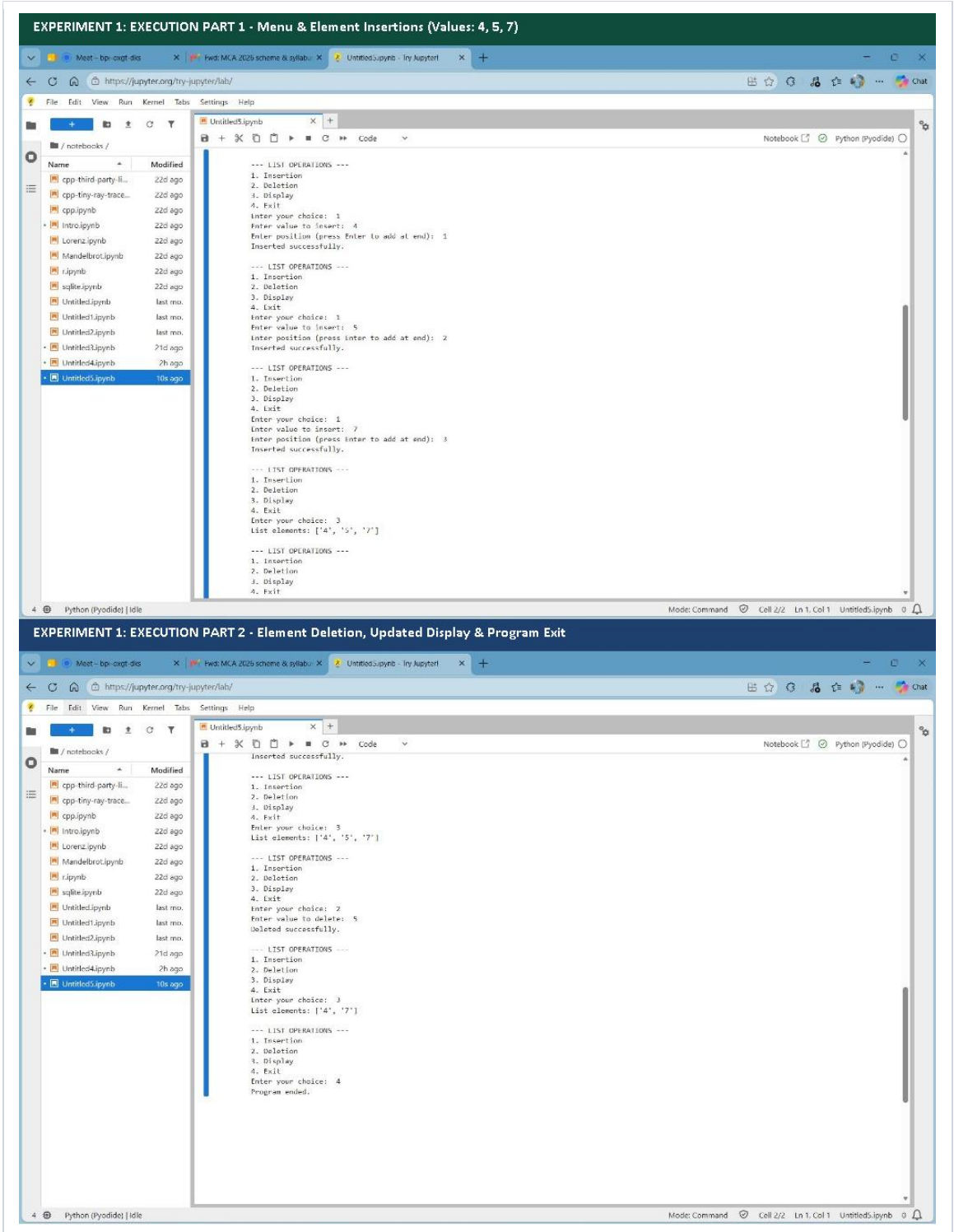


Figure: Execution output of List Operations (Insertion, Deletion and Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- LIST OPERATIONS ---
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 4
Enter position (press Enter to add at end): 1
Inserted successfully.

--- LIST OPERATIONS ---
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 5
Enter position (press Enter to add at end): 2
Inserted successfully.

--- LIST OPERATIONS ---
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 7
Enter position (press Enter to add at end): 3
Inserted successfully.

--- LIST OPERATIONS ---
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3
List elements: ['4', '5', '7']

--- LIST OPERATIONS ---
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 2
Enter value to delete: 5
Deleted successfully.

--- LIST OPERATIONS ---
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3
List elements: ['4', '7']

--- LIST OPERATIONS ---
```

```
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 4
Program ended.
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 2: SORTING ALGORITHMS (BUBBLE, QUICK, INSERTION, MERGE, HEAP) & TIME COMPARISON

AIM: To write and execute a Python program to sort a list using Bubble Sort, Insertion Sort, Quick Sort, Merge Sort, and Heap Sort, and compare them on the basis of execution time and asymptotic time complexities.

Algorithmic Comparison Matrix

Algorithm	Best Time	Avg Time	Worst Time	Aux Space	Stability	In-Place
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Yes
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Yes

Python Source Code

```
# Sorting a list using Bubble, Quick, Insertion, Merge and Heap sort
# and comparing them on the basis of time

import random
import time

def bubble_sort(a):
    n = len(a)
    for i in range(n):
        for j in range(n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a

def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a

def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[len(a) // 2]
```

```
    left = [x for x in a if x < pivot]
    middle = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    while left and right:
        if left[0] <= right[0]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))
    return result + left + right

def heapify(a, n, i):
    largest = i
    l, r = 2 * i + 1, 2 * i + 2
    if l < n and a[l] > a[largest]:
        largest = l
    if r < n and a[r] > a[largest]:
        largest = r
    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        heapify(a, n, largest)

def heap_sort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)
    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(a, i, 0)
    return a

# Main program
data = [random.randint(1, 10000) for _ in range(2000)]
print("Original list (first 10):", data[:10])
print("Sorted list   (first 10):", sorted(data)[:10])

algorithms = [
    ("Bubble Sort",    bubble_sort,    "O(n)",       "O(n^2)",     "O(n^2)"),
    ("Insertion Sort", insertion_sort, "O(n)",       "O(n^2)",     "O(n^2)"),
    ("Quick Sort",     quick_sort,     "O(n log n)", "O(n log n)", "O(n^2)"),
    ("Merge Sort",     merge_sort,     "O(n log n)", "O(n log n)", "O(n log n)"),
    ("Heap Sort",      heap_sort,      "O(n log n)", "O(n log n)", "O(n log n)"),
]

print("\n%-15s %-12s %-12s %-12s %s" % ("Algorithm", "Best", "Average", "Worst", "Time taken
(sec)"))
print("-" * 70)
for name, func, best, avg, worst in algorithms:
    start = time.time()
```

```
    result = func(data.copy())
    end = time.time()
    assert result == sorted(data)
    print("%-15s %-12s %-12s %-12s %.4f" % (name, best, avg, worst, end - start))
```

Space for Output

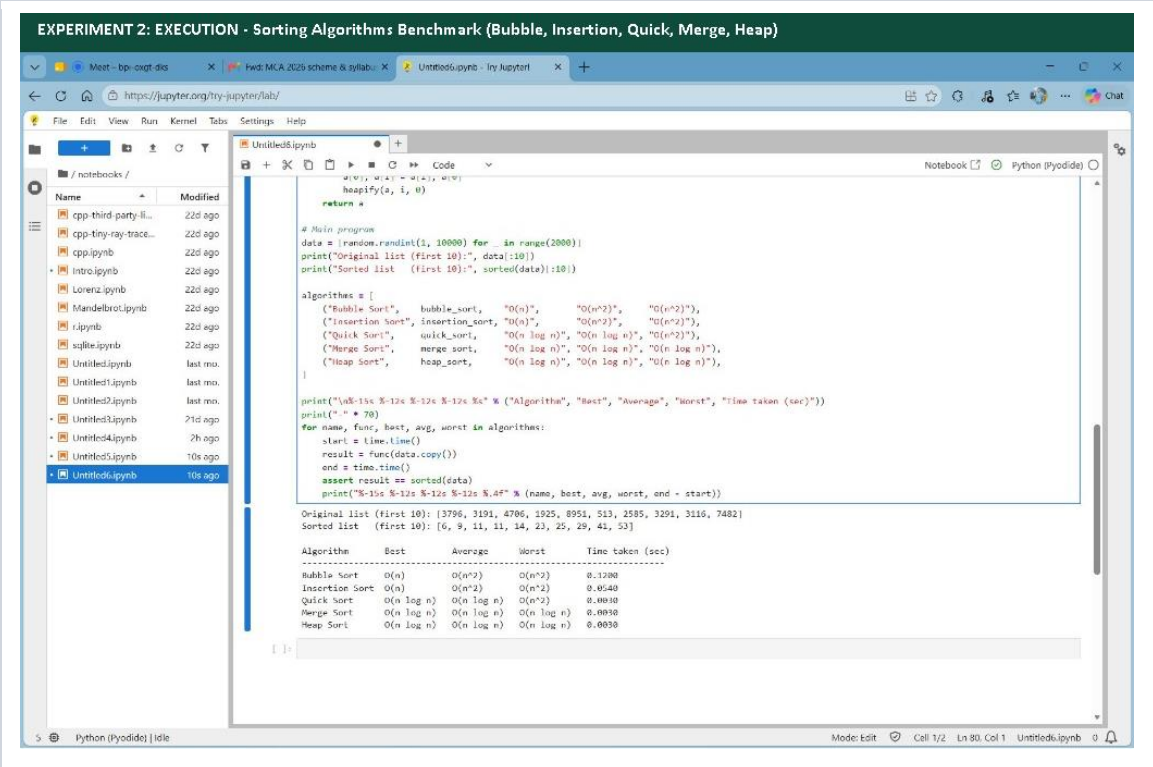


Figure: Execution output of Sorting Algorithms & Time Complexity Comparison

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Original list (first 10): [3796, 3191, 4706, 1925, 8951, 513, 2585, 3291, 3116, 7482]
Sorted list   (first 10): [6, 9, 11, 11, 14, 23, 25, 29, 41, 53]

Algorithm       Best         Average      Worst        Time taken (sec)
-----------------------------------------------------------------------
Bubble Sort     O(n)         O(n^2)       O(n^2)       0.1200
Insertion Sort  O(n)         O(n^2)       O(n^2)       0.0540
Quick Sort      O(n log n)   O(n log n)   O(n^2)       0.0030
Merge Sort      O(n log n)   O(n log n)   O(n log n)   0.0030
Heap Sort       O(n log n)   O(n log n)   O(n log n)   0.0030
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 3: SEARCHING ALGORITHMS (LINEAR SEARCH AND BINARY SEARCH) & TIME COMPARISON

AIM: To write and execute a Python program to search for an element using Linear Search and Binary Search algorithms and compare their efficiency on the basis of execution time and asymptotic time complexities.

Searching Algorithms Comparison Matrix

Algorithm	Best Time	Average Time	Worst Time	Aux Space	Sorted Data Required?
Linear Search	O(1)	O(n)	O(n)	O(1)	No (Works on any list)
Binary Search	O(1)	O(log n)	O(log n)	O(1)	Yes (Requires sorted list)

Python Source Code

```
# Searching an element using Linear Search and Binary Search
# and comparing them on the basis of time

import time

def linear_search(a, key):
    for i in range(len(a)):
        if a[i] == key:
            return i
    return -1

def binary_search(a, key):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] == key:
            return mid
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Main program
a = list(range(1, 1000001))      # sorted list of 1 million numbers
key = int(input("Enter element to search (1 to 1000000): "))

start = time.time()
pos1 = linear_search(a, key)
t1 = time.time() - start

start = time.time()
pos2 = binary_search(a, key)
t2 = time.time() - start
```

```
print("\nLinear Search : found at index", pos1, "| time = %.8f sec" % t1)
print("Binary Search : found at index", pos2, "| time = %.8f sec" % t2)

print("\n%-15s %-8s %-10s %-10s %s" % ("Algorithm", "Best", "Average", "Worst", "Sorted list
needed?"))
print("-" * 60)
print("%-15s %-8s %-10s %-10s %s" % ("Linear Search", "O(1)", "O(n)", "O(n)", "No"))
print("%-15s %-8s %-10s %-10s %s" % ("Binary Search", "O(1)", "O(log n)", "O(log n)", "Yes"))
```

Space for Output

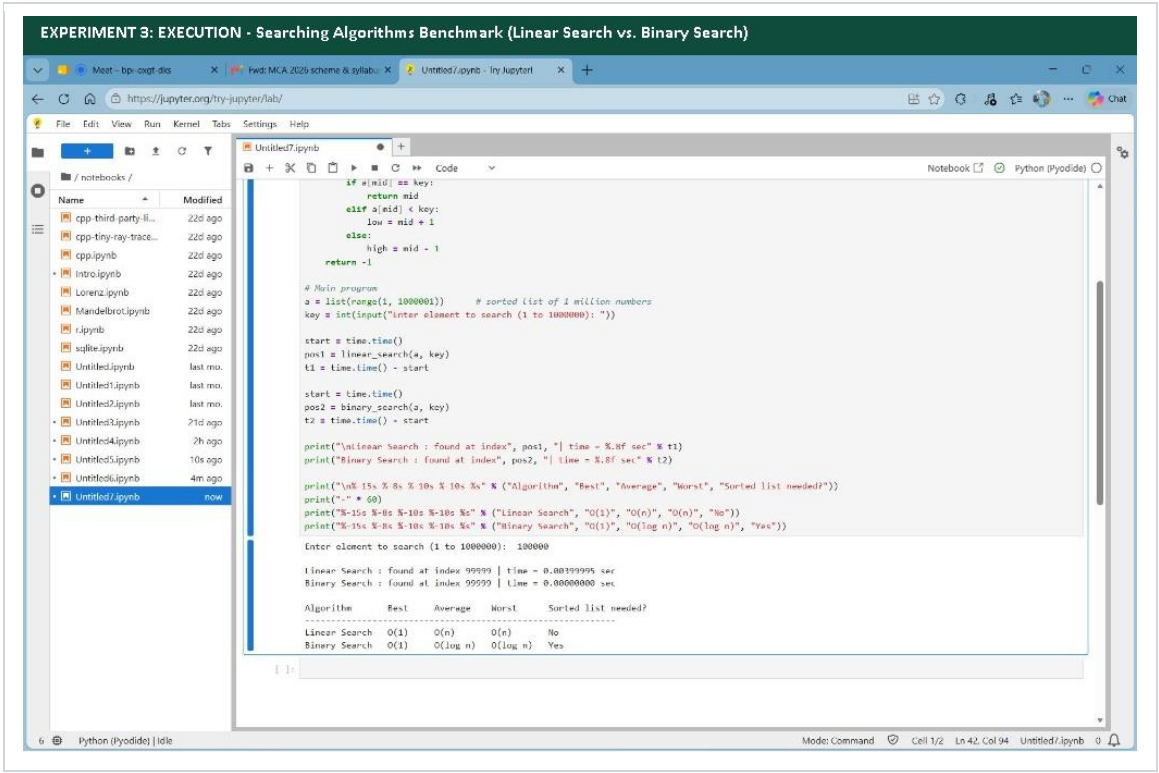


Figure: Execution output of Linear Search vs Binary Search & Time Comparison

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter element to search (1 to 1000000): 100000

Linear Search: found at index 99999 | time = 0.00399995 sec
Binary Search: found at index 99999 | time = 0.00000000 sec

Algorithm       Best     Average    Worst      Sorted list needed?
-----------------------------------------------------------
Linear Search   O(1)     O(n)       O(n)       No
Binary Search   O(1)     O(log n)   O(log n)   Yes
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 4: STACK, QUEUE, CIRCULAR QUEUE (USING ARRAYS) & BROWSER HISTORY (USING STACKS)

AIM: To implement Stack, Linear Queue, and Circular Queue data structures using fixed-size arrays, and simulate web browser navigation history using dual stacks in Python.

Python Source Code

```
# Stack, Queue, Circular Queue (using arrays) and Browser History (using stacks)

SIZE = 5

# ---------------- STACK ----------------
class Stack:
    def __init__(self):
        self.arr = [None] * SIZE
        self.top = -1

    def push(self, x):
        if self.top == SIZE - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.arr[self.top] = x
            print(x, "pushed")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print(self.arr[self.top], "popped")
            self.top -= 1

    def peek(self):
        print("Top element:", self.arr[self.top]) if self.top != -1 else print("Stack is empty")

    def display(self):
        print("Stack:", self.arr[:self.top + 1]) if self.top != -1 else print("Stack is empty")

# ---------------- QUEUE ----------------
class Queue:
    def __init__(self):
        self.arr = [None] * SIZE
        self.front = 0
        self.rear = -1

    def enqueue(self, x):
        if self.rear == SIZE - 1:
            print("Queue is Full")
        else:
            self.rear += 1
            self.arr[self.rear] = x
            print(x, "inserted")

    def dequeue(self):
        if self.front > self.rear:
```

```
            print("Queue is Empty")
        else:
            print(self.arr[self.front], "removed")
            self.front += 1

    def display(self):
        print("Queue:", self.arr[self.front:self.rear + 1]) if self.front <= self.rear else
print("Queue is empty")

# ---------------- CIRCULAR QUEUE ----------------
class CircularQueue:
    def __init__(self):
        self.arr = [None] * SIZE
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if (self.rear + 1) % SIZE == self.front:
            print("Circular Queue is Full")
            return
        if self.front == -1:
            self.front = 0
        self.rear = (self.rear + 1) % SIZE
        self.arr[self.rear] = x
        print(x, "inserted")

    def dequeue(self):
        if self.front == -1:
            print("Circular Queue is Empty")
            return
        print(self.arr[self.front], "removed")
        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % SIZE

    def display(self):
        if self.front == -1:
            print("Circular Queue is empty")
            return
        items = []
        i = self.front
        while True:
            items.append(self.arr[i])
            if i == self.rear:
                break
            i = (i + 1) % SIZE
        print("Circular Queue:", items)

# ---------------- BROWSER HISTORY (2 stacks) ----------------
class Browser:
    def __init__(self):
        self.back = []        # stack of previous pages
        self.forward = []     # stack of forward pages
        self.current = "Home"

    def visit(self, url):
```

```
        self.back.append(self.current)
        self.current = url
        self.forward = []     # new visit clears forward history
        print("Visited:", self.current)

    def go_back(self):
        if not self.back:
            print("No page to go back")
        else:
            self.forward.append(self.current)
            self.current = self.back.pop()
            print("Back to:", self.current)

    def go_forward(self):
        if not self.forward:
            print("No page to go forward")
        else:
            self.back.append(self.current)
            self.current = self.forward.pop()
            print("Forward to:", self.current)

    def show(self):
        print("Back stack   :", self.back)
        print("Current page :", self.current)
        print("Forward stack:", self.forward)

# ---------------- MAIN MENU ----------------
s, q, cq, b = Stack(), Queue(), CircularQueue(), Browser()

while True:
    print("\n1.Stack  2.Queue  3.Circular Queue  4.Browser History  5.Exit")
    ch = input("Enter choice: ")

    if ch == "1":
        while True:
            c = input("\n[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: ")
            if c == "1": s.push(input("Enter value: "))
            elif c == "2": s.pop()
            elif c == "3": s.peek()
            elif c == "4": s.display()
            else: break

    elif ch == "2":
        while True:
            c = input("\n[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: ")
            if c == "1": q.enqueue(input("Enter value: "))
            elif c == "2": q.dequeue()
            elif c == "3": q.display()
            else: break

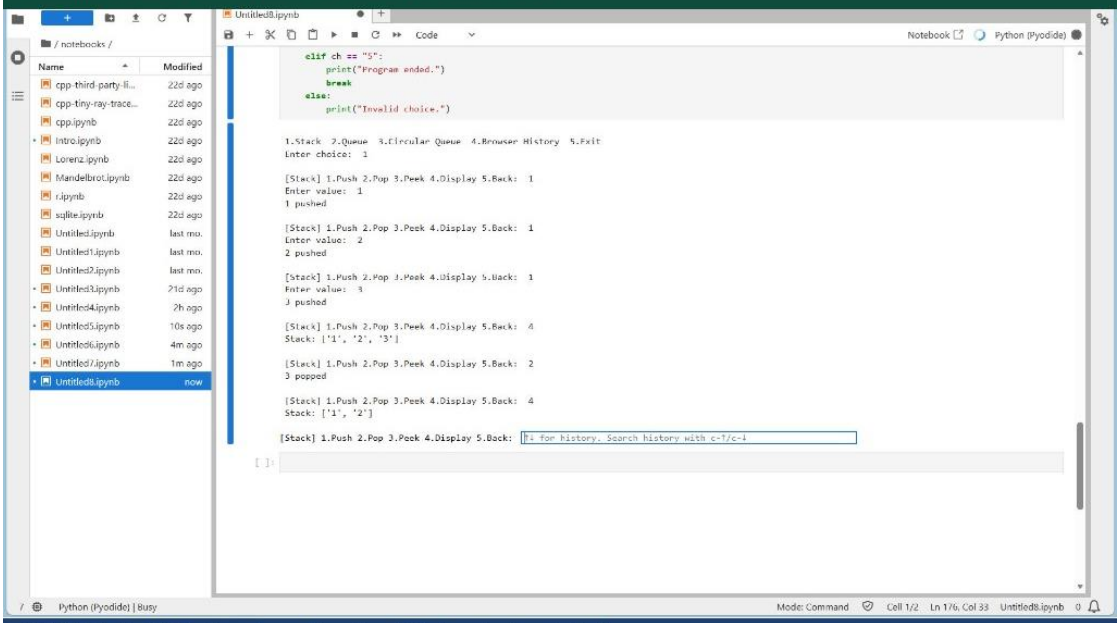
    elif ch == "3":
        while True:
            c = input("\n[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: ")
            if c == "1": cq.enqueue(input("Enter value: "))
            elif c == "2": cq.dequeue()
            elif c == "3": cq.display()
            else: break
```

```
    elif ch == "4":
        while True:
            c = input("\n[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: ")
            if c == "1": b.visit(input("Enter website: "))
            elif c == "2": b.go_back()
            elif c == "3": b.go_forward()
            elif c == "4": b.show()
            else: break

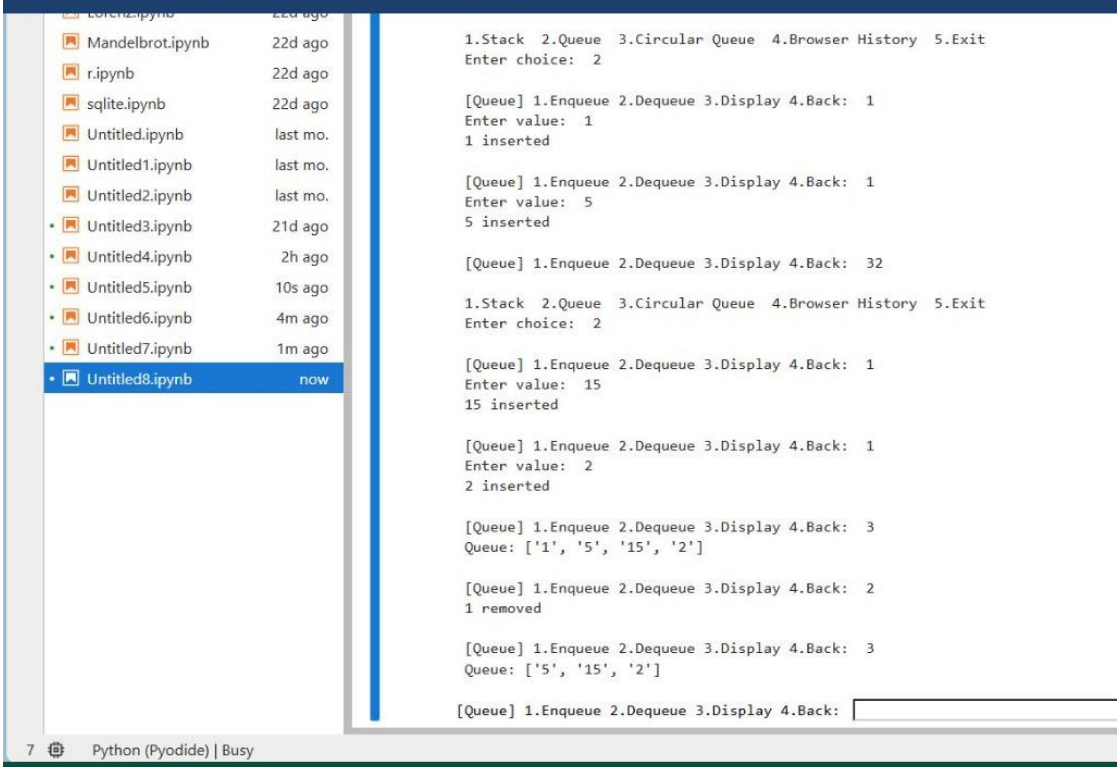
    elif ch == "5":
        print("Program ended.")
        break
    else:
        print("Invalid choice.")
```

Space for Output

EXPERIMENT 4 - PART 1: Stack Operations (LIFO Array Implementation)



EXPERIMENT 4 - PART 2: Linear Queue Operations (FIFO Array Implementation)



EXPERIMENT 4 - PART 3: Circular Queue Operations (Ring Buffer with Modulo Arithmetic)

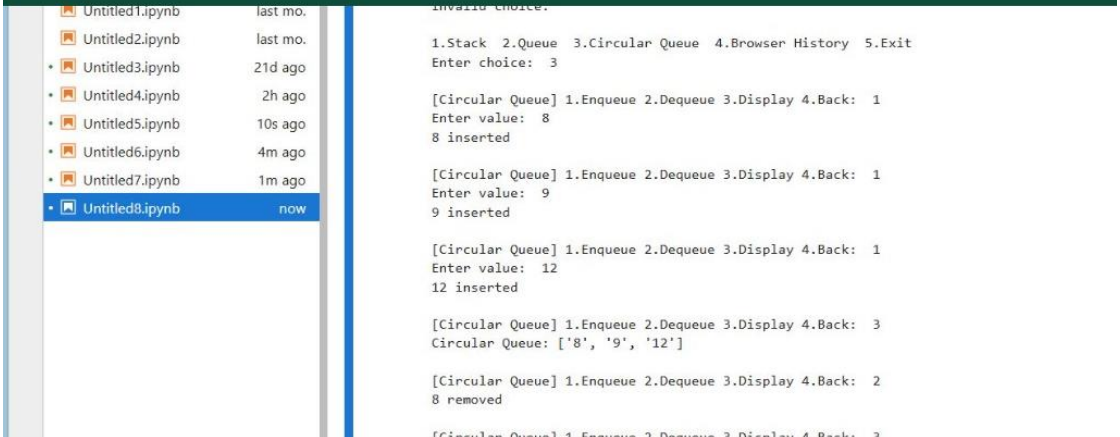


Figure: Execution output of Stack, Queue, Circular Queue and Browser History

Sample Run: Execution Log / Console Output (Terminal Execution):

```
1.Stack  2.Queue  3.Circular Queue  4.Browser History  5.Exit
Enter choice: 1

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Enter value: 1
1 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Enter value: 2
2 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Enter value: 3
3 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 4
Stack: ['1', '2', '3']

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 2
3 popped

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 4
Stack: ['1', '2']

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 5

1.Stack  2.Queue  3.Circular Queue  4.Browser History  5.Exit
Enter choice: 2

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 1
1 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 5
5 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 15
15 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 2
2 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Queue: ['1', '5', '15', '2']

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 2
1 removed

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Queue: ['5', '15', '2']
```

```
[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 4

1.Stack  2.Queue  3.Circular Queue  4.Browser History  5.Exit
Enter choice: 3

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 8
8 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 9
9 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 12
12 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Circular Queue: ['8', '9', '12']

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 2
8 removed

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Circular Queue: ['9', '12']

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 4

1.Stack  2.Queue  3.Circular Queue  4.Browser History  5.Exit
Enter choice: 4

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 1
Enter website: https://www.youtube.com/
Visited: https://www.youtube.com/

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 1
Enter website: https://www.amazon.in/
Visited: https://www.amazon.in/

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 4
Back stack   : ['Home', 'https://www.youtube.com/']
Current page : https://www.amazon.in/
Forward stack: []

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 3
No page to go forward

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 2
Back to: https://www.youtube.com/

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 3
Forward to: https://www.amazon.in/

[Browser] 1.Visit 2.Back 3.Forward 4.Show 5.Exit: 5

1.Stack  2.Queue  3.Circular Queue  4.Browser History  5.Exit
```

```
Enter choice: 5
Program ended.
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 5: EVALUATION OF A POSTFIX EXPRESSION USING A STACK

AIM: To write and execute a Python program to evaluate a Postfix (Reverse Polish) expression using the stack data structure.

Python Source Code

```
# Evaluation of a Postfix Expression using a Stack

def evaluate_postfix(expression):
    stack = []
    for token in expression.split():
        if token.isdigit():                # operand -> push
            stack.append(int(token))
        else:                              # operator -> pop two, calculate, push
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                stack.append(a / b)
            elif token == '^':
                stack.append(a ** b)
    return stack.pop()

# Main program
exp = input("Enter postfix expression (separate with spaces, e.g. 2 3 1 * + 9 -): ")
print("Result =", evaluate_postfix(exp))
```

Space for Output

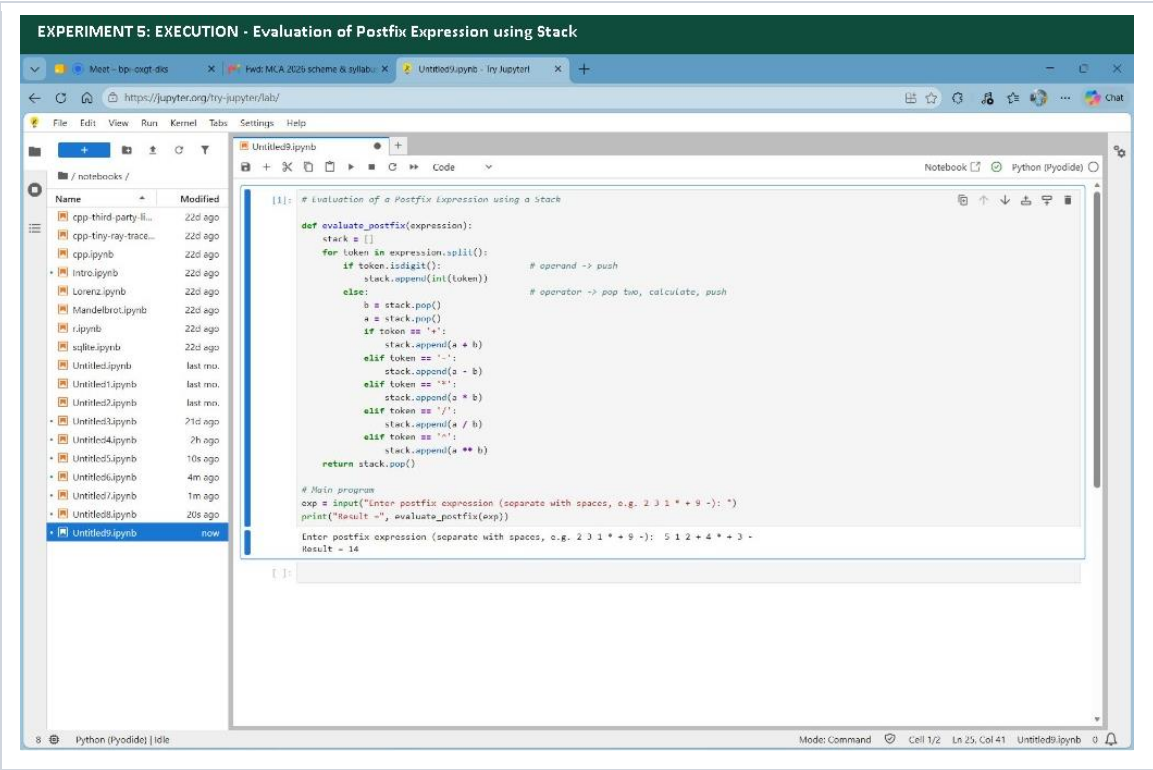


Figure: Execution output of Evaluation of Postfix Expression

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter postfix expression (separate with spaces, e.g. 2 3 1 * + 9 -): 5 1 2 + 4 * + 3 -
Result = 14
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 6: CONVERSION OF INFIX EXPRESSION TO POSTFIX USING STACK

AIM: To write and execute a Python program that converts an Infix mathematical expression to a Postfix expression using the stack-based Shunting-Yard algorithm.

Python Source Code

```
# Conversion of Infix Expression to Postfix Expression using a Stack

def precedence(op):
    if op == '^':
        return 3
    if op in '*/':
        return 2
    if op in '+-':
        return 1
    return 0

def infix_to_postfix(expression):
    stack = []
    result = ""
    for ch in expression:
        if ch.isalnum():                    # operand -> add to result
            result += ch
        elif ch == '(':
            stack.append(ch)
        elif ch == ')':
            while stack and stack[-1] != '(':
                result += stack.pop()
            stack.pop()                     # remove '('
        else:                               # operator
            while stack and precedence(stack[-1]) >= precedence(ch) and ch != '^':
                result += stack.pop()
            stack.append(ch)
    while stack:
        result += stack.pop()
    return result

# Main program
exp = input("Enter infix expression (e.g. A+B*C): ").replace(" ", "")
print("Postfix expression =", infix_to_postfix(exp))
```

Space for Output

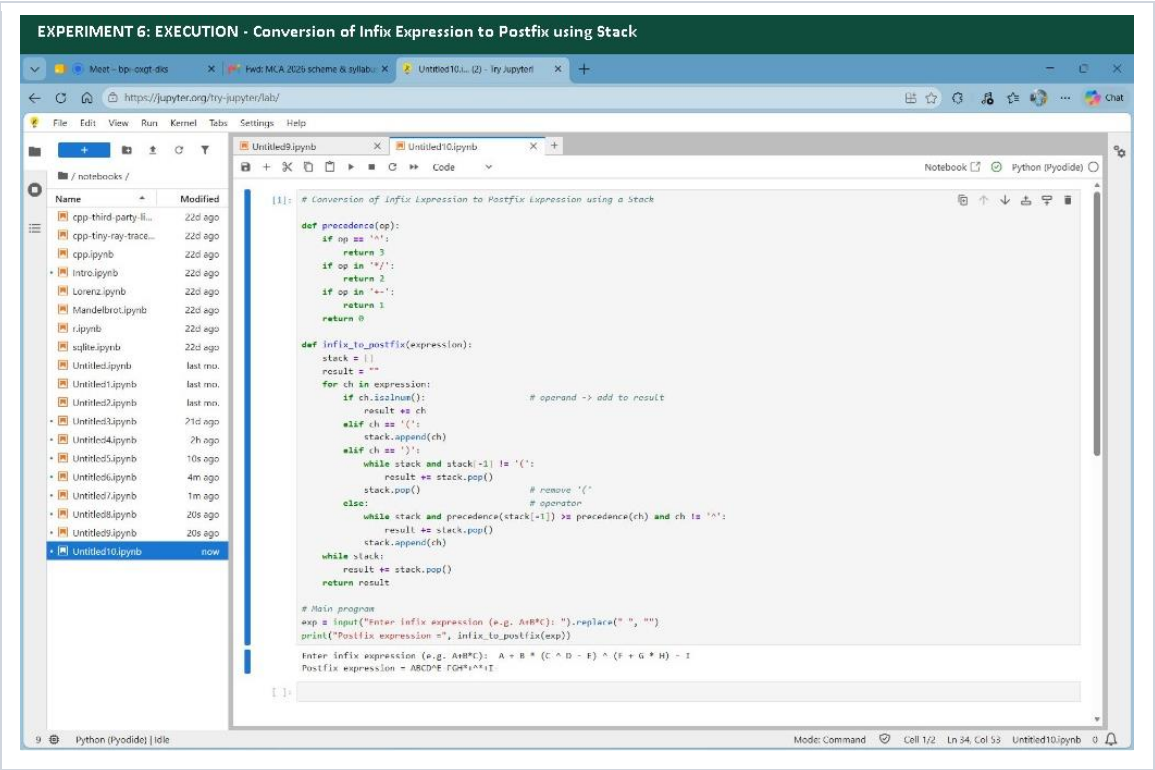


Figure: Execution output of Conversion of Infix to Postfix

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter infix expression (e.g. A+B*C): A + B * (C ^ D - E) ^ (F + G * H) - I
Postfix expression = ABCD^E-FGH*+^*+I-
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 7: SINGLY LINKED LIST OPERATIONS (INSERTION, DELETION, DISPLAY)

AIM: To write and execute a Python program to implement a Singly Linked List supporting insertion at the beginning and end, deletion of a node by value, and displaying the list.

Python Source Code

```
# Singly Linked List: Insertion, Deletion, Display

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        node = Node(data)
        node.next = self.head
        self.head = node
        print("Inserted at beginning.")

    def insert_end(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = node
        print("Inserted at end.")

    def delete(self, data):
        if self.head is None:
            print("List is empty.")
            return
        if self.head.data == data:
            self.head = self.head.next
            print("Deleted successfully.")
            return
        temp = self.head
        while temp.next and temp.next.data != data:
            temp = temp.next
        if temp.next is None:
            print("Value not found.")
        else:
            temp.next = temp.next.next
            print("Deleted successfully.")

    def display(self):
        if self.head is None:
            print("List is empty.")
            return
```

```
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

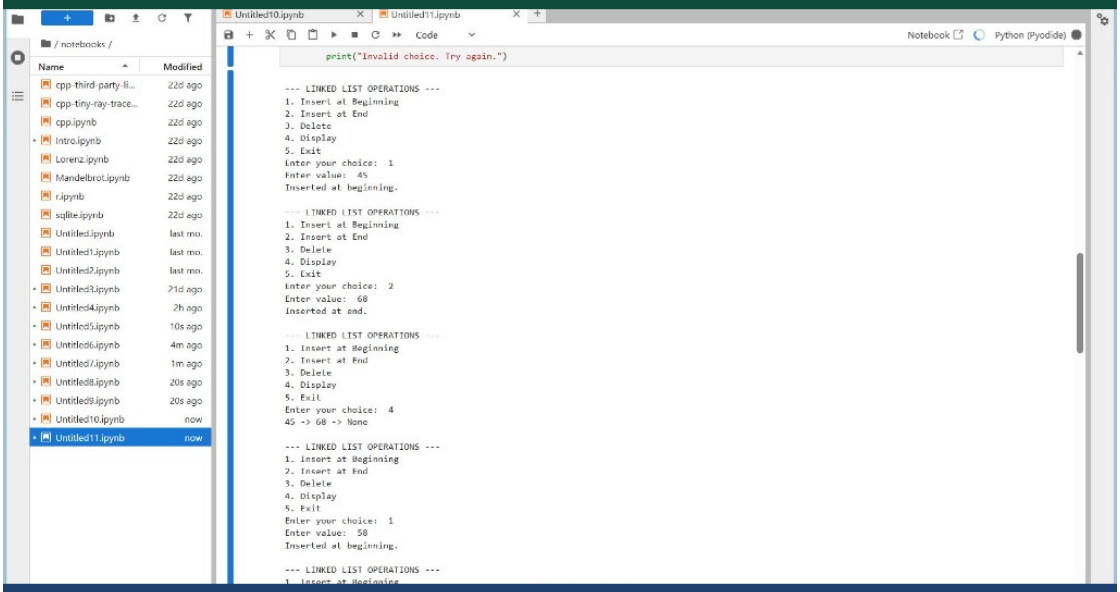
# Main program
ll = LinkedList()

while True:
    print("\n--- LINKED LIST OPERATIONS ---")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    choice = input("Enter your choice: ")

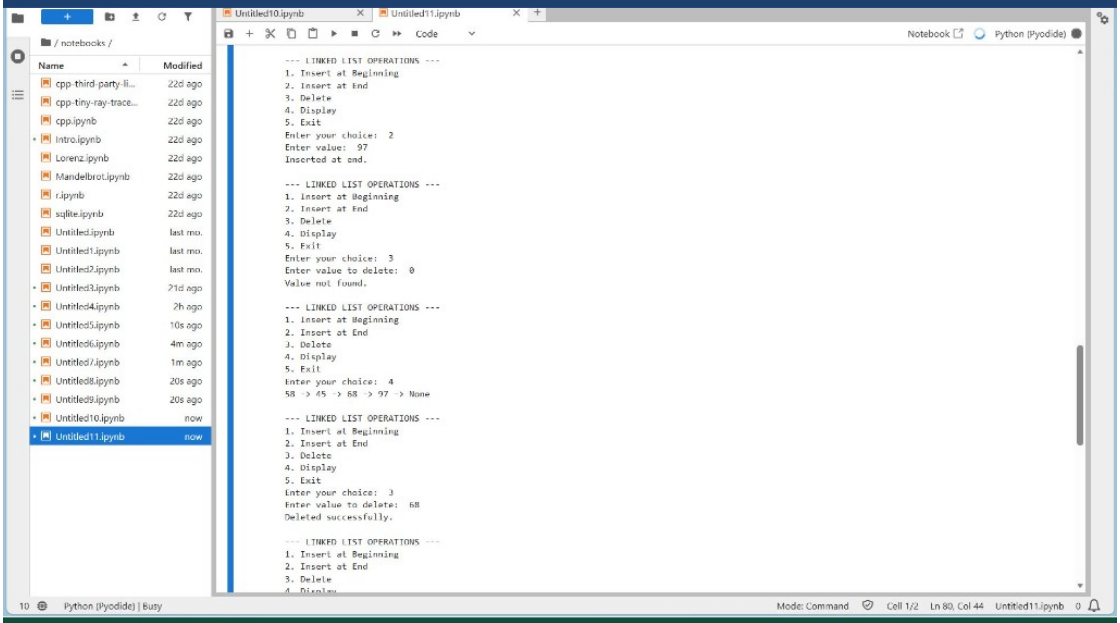
    if choice == "1":
        ll.insert_begin(input("Enter value: "))
    elif choice == "2":
        ll.insert_end(input("Enter value: "))
    elif choice == "3":
        ll.delete(input("Enter value to delete: "))
    elif choice == "4":
        ll.display()
    elif choice == "5":
        print("Program ended.")
        break
    else:
        print("Invalid choice. Try again.")
```

Space for Output

EXPERIMENT 7 - PART 1: Node Insertions at Beginning and End (Values: 45, 68, 58)



EXPERIMENT 7 - PART 2: Insertion of Node 97, Deletion Validation (0), and Deletion of Node 68



EXPERIMENT 7 - PART 3: Final Verification Display of Modified Linked List (58 -> 45 -> 97)

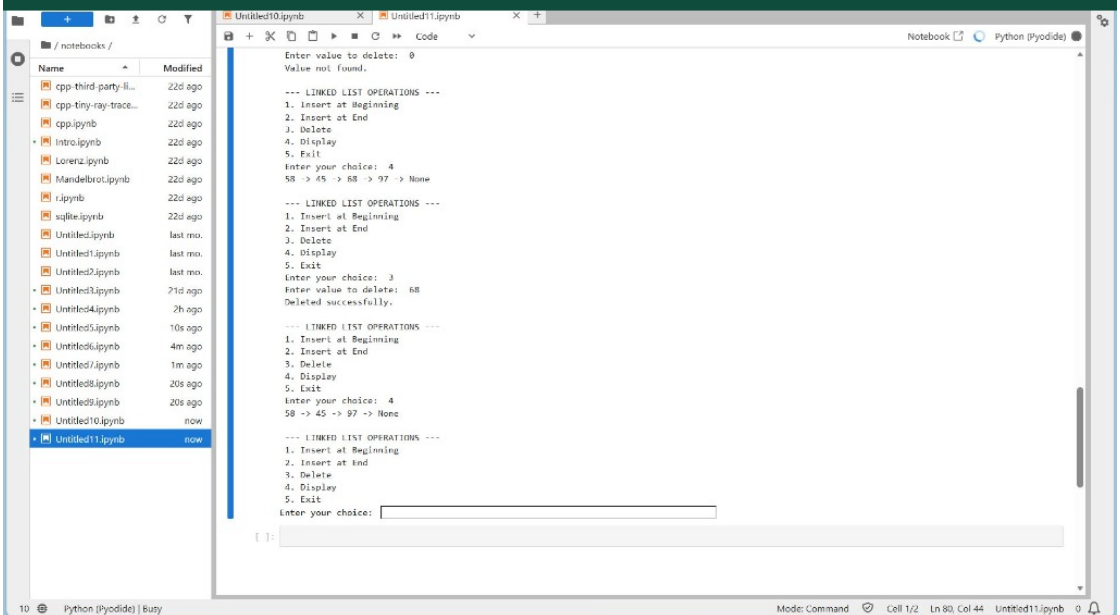


Figure: Execution output of Singly Linked List (Insert, Delete, Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter value: 45
Inserted at beginning.

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter value: 68
Inserted at end.

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 4
45 -> 68 -> None

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter value: 58
Inserted at beginning.

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter value: 97
Inserted at end.

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
```

```
4. Display
5. Exit
Enter your choice: 3
Enter value to delete: 0
Value not found.

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 4
58 -> 45 -> 68 -> 97 -> None

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 3
Enter value to delete: 68
Deleted successfully.

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 4
58 -> 45 -> 97 -> None

--- LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 5
Program ended.
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 8: CIRCULAR LINKED LIST OPERATIONS (INSERTION, DELETION, DISPLAY)

AIM: To write and execute a Python program to implement a Circular Singly Linked List supporting insertion at the beginning and end, deletion of nodes by value, and displaying the circular structure.

Python Source Code

```
# Circular Linked List: Insertion, Deletion, Display

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        node = Node(data)
        if self.head is None:
            node.next = node                # points to itself
        else:
            last = self.head
            while last.next != self.head:
                last = last.next
            node.next = self.head
            last.next = node
        self.head = node
        print("Inserted at beginning.")

    def insert_end(self, data):
        node = Node(data)
        if self.head is None:
            node.next = node
            self.head = node
        else:
            last = self.head
            while last.next != self.head:
                last = last.next
            last.next = node
            node.next = self.head
        print("Inserted at end.")

    def delete(self, data):
        if self.head is None:
            print("List is empty.")
            return
        # only one node in the list
        if self.head.next == self.head:
            if self.head.data == data:
                self.head = None
                print("Deleted successfully.")
            else:
                print("Value not found.")
```

```
            return
        # deleting the head node
        if self.head.data == data:
            last = self.head
            while last.next != self.head:
                last = last.next
            self.head = self.head.next
            last.next = self.head
            print("Deleted successfully.")
            return
        # deleting any other node
        prev = self.head
        while prev.next != self.head and prev.next.data != data:
            prev = prev.next
        if prev.next == self.head:
            print("Value not found.")
        else:
            prev.next = prev.next.next
            print("Deleted successfully.")

    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")

# Main program
cll = CircularLinkedList()

while True:
    print("
--- CIRCULAR LINKED LIST OPERATIONS ---")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        cll.insert_begin(input("Enter value: "))
    elif choice == "2":
        cll.insert_end(input("Enter value: "))
    elif choice == "3":
        cll.delete(input("Enter value to delete: "))
    elif choice == "4":
        cll.display()
    elif choice == "5":
        print("Program ended.")
        break
    else:
```

```
        print("Invalid choice. Try again.")
```

Space for Output

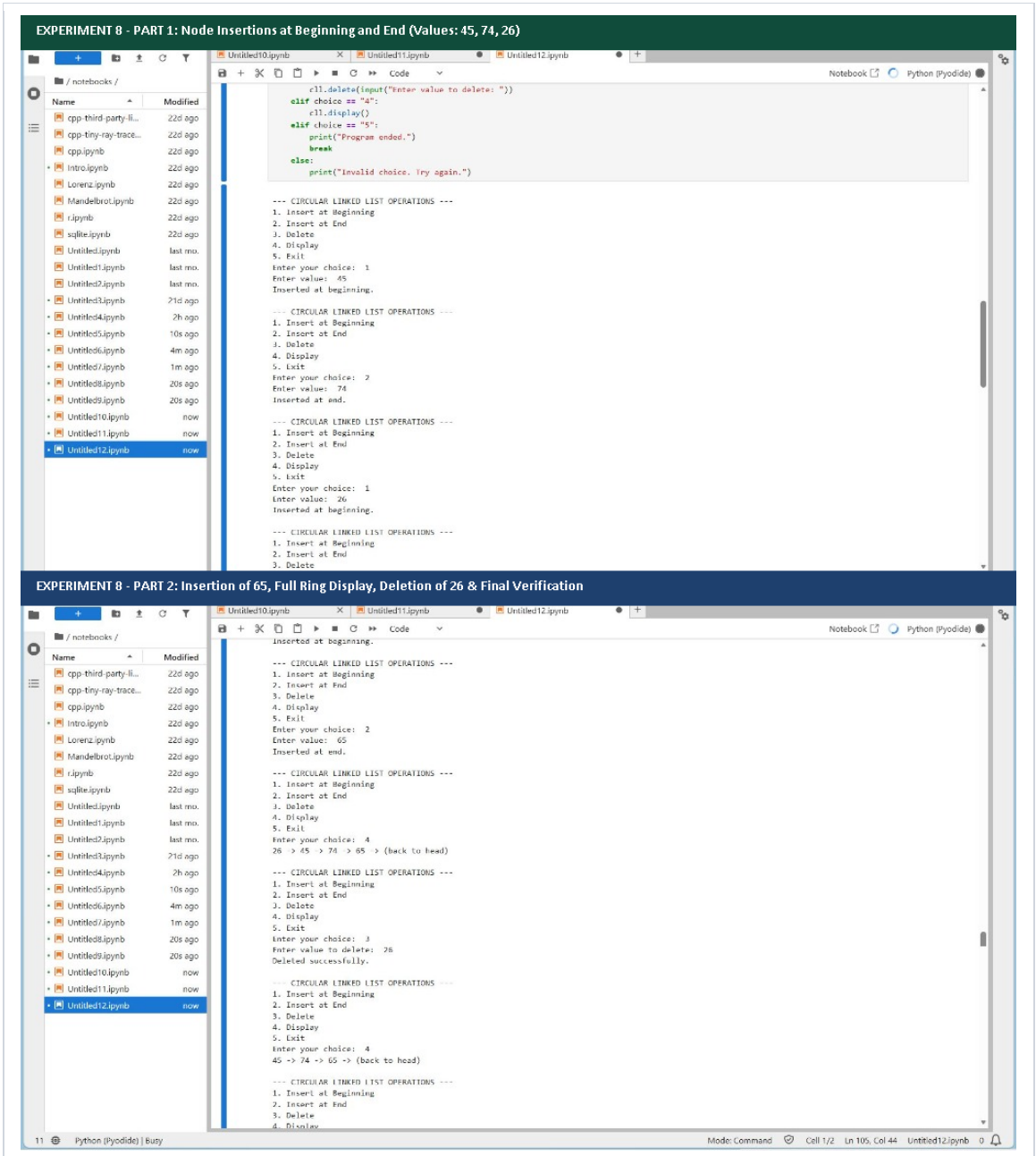


Figure: Execution output of Circular Linked List (Insert, Delete, Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
```

```
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter value: 45
Inserted at beginning.

--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter value: 74
Inserted at end.

--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter value: 26
Inserted at beginning.

--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter value: 65
Inserted at end.

--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 4
26 -> 45 -> 74 -> 65 -> (back to head)

--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 3
Enter value to delete: 26
Deleted successfully.

--- CIRCULAR LINKED LIST OPERATIONS ---
```

```
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 4
45 -> 74 -> 65 -> (back to head)

--- CIRCULAR LINKED LIST OPERATIONS ---
1. Insert at Beginning
2. Insert at End
3. Delete
4. Display
5. Exit
Enter your choice: 5
Program ended.
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 9: STACK, QUEUE, AND CIRCULAR QUEUE IMPLEMENTATION USING LINKED LIST

AIM: To write and execute a Python program to implement Stack, Queue, and Circular Queue data structures using Singly Linked Lists and Circular Linked Lists.

Python Source Code

```
# Stack, Queue and Circular Queue using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# ---------------- STACK (insert/remove at head) ----------------
class Stack:
    def __init__(self):
        self.top = None

    def push(self, x):
        node = Node(x)
        node.next = self.top
        self.top = node
        print(x, "pushed")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print(self.top.data, "popped")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
            return
        temp = self.top
        print("Stack (top to bottom):", end=" ")
        while temp:
            print(temp.data, end=" ")
            temp = temp.next
        print()

# ---------------- QUEUE (insert at rear, remove from front) ----------------
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
```

```
    def enqueue(self, x):
        node = Node(x)
        if self.rear is None:
            self.front = self.rear = node
        else:
            self.rear.next = node
            self.rear = node
        print(x, "inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue is Empty")
            return
        print(self.front.data, "removed")
        self.front = self.front.next
        if self.front is None:
            self.rear = None

    def display(self):
        if self.front is None:
            print("Queue is empty")
            return
        temp = self.front
        print("Queue (front to rear):", end=" ")
        while temp:
            print(temp.data, end=" ")
            temp = temp.next
        print()

# ---------------- CIRCULAR QUEUE (rear.next always points to front) ----------------
class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, x):
        node = Node(x)
        if self.front is None:
            self.front = self.rear = node
        else:
            self.rear.next = node
            self.rear = node
        self.rear.next = self.front         # keep the circle
        print(x, "inserted")

    def dequeue(self):
        if self.front is None:
            print("Circular Queue is Empty")
            return
        print(self.front.data, "removed")
        if self.front == self.rear:         # only one node
            self.front = self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front

    def display(self):
```

```
        if self.front is None:
            print("Circular Queue is empty")
            return
        temp = self.front
        print("Circular Queue:", end=" ")
        while True:
            print(temp.data, end=" ")
            temp = temp.next
            if temp == self.front:
                break
        print("(back to front)")

# ---------------- MAIN MENU ----------------
s, q, cq = Stack(), Queue(), CircularQueue()

while True:
    print("\n1.Stack  2.Queue  3.Circular Queue  4.Exit")
    ch = input("Enter choice: ")

    if ch == "1":
        while True:
            c = input("\n[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: ")
            if c == "1": s.push(input("Enter value: "))
            elif c == "2": s.pop()
            elif c == "3": s.peek()
            elif c == "4": s.display()
            else: break

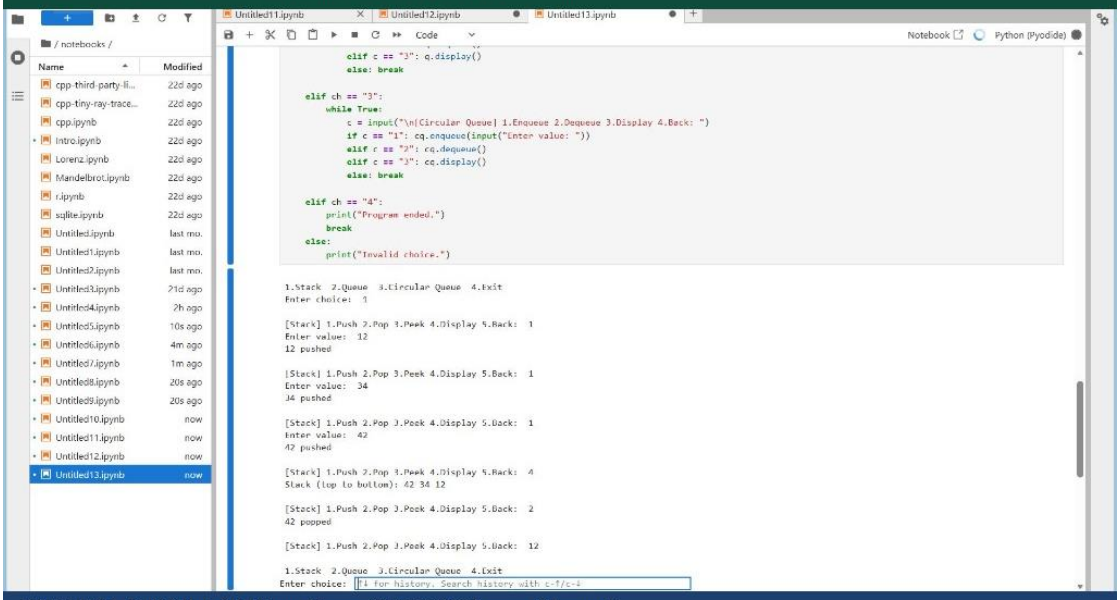
    elif ch == "2":
        while True:
            c = input("\n[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: ")
            if c == "1": q.enqueue(input("Enter value: "))
            elif c == "2": q.dequeue()
            elif c == "3": q.display()
            else: break

    elif ch == "3":
        while True:
            c = input("\n[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: ")
            if c == "1": cq.enqueue(input("Enter value: "))
            elif c == "2": cq.dequeue()
            elif c == "3": cq.display()
            else: break

    elif ch == "4":
        print("Program ended.")
        break
    else:
        print("Invalid choice.")
```

Space for Output

EXPERIMENT 9 - PART 1: Linked List Stack (Push 12, 34, 42 | Peek & Pop Verification)



```
            elif c == "3": q.display()
            else: break

    elif ch == "3":
        while True:
            c = input("\n[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: ")
            if c == "1": cq.enqueue(input("Enter value: "))
            elif c == "2": cq.dequeue()
            elif c == "3": cq.display()
            else: break

    elif ch == "4":
        print("Program ended.")
        break
    else:
        print("Invalid choice.")
```

```
1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice:  1

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back:  1
Enter value:  12
12 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back:  1
Enter value:  34
34 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back:  1
Enter value:  42
42 pushed

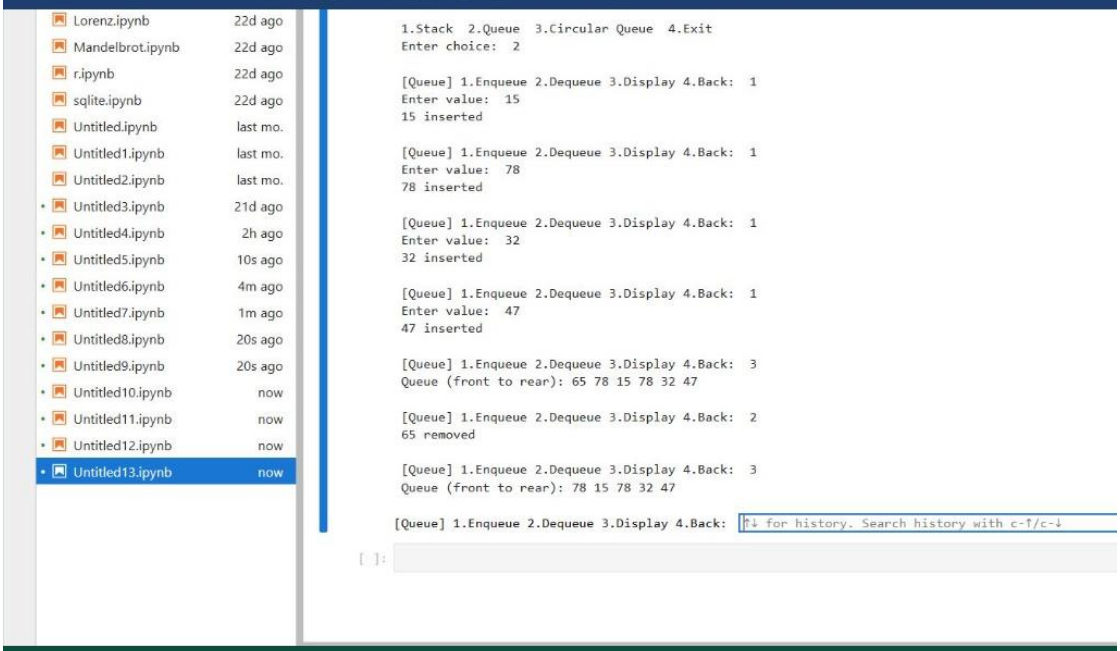
[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back:  4
Stack (top to bottom): 42 34 12

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back:  2
42 popped

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back:  12

1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice:  ↑↓ for history. Search history with c-↑/c-↓
```

EXPERIMENT 9 - PART 2: Linked List Queue (Enqueue 15, 78, 32, 47 | Dequeue & Traversal)



```
1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice:  2

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  15
15 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  78
78 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  32
32 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  47
47 inserted

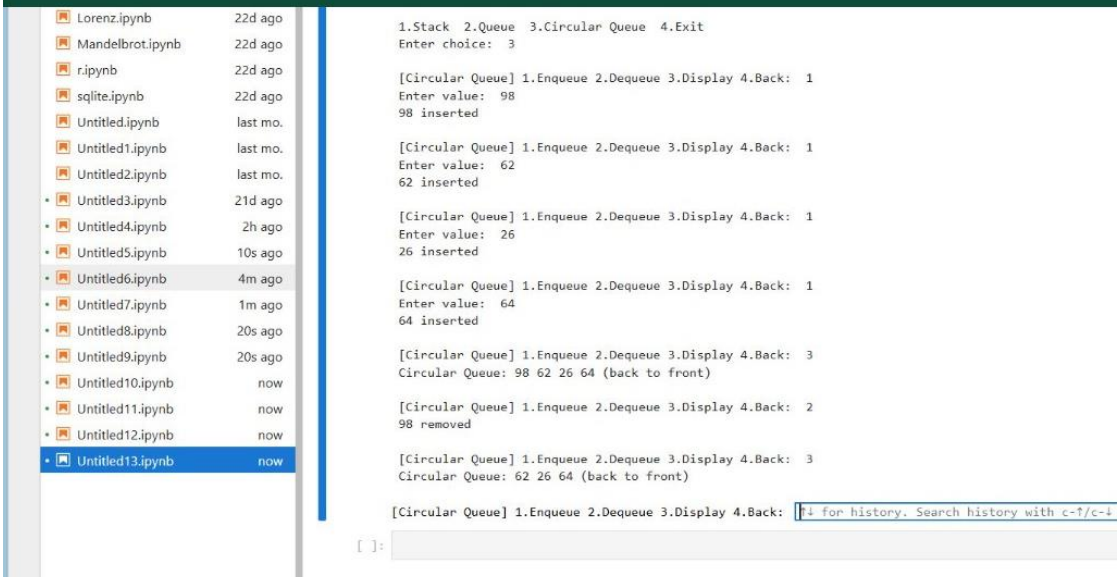
[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  3
Queue (front to rear): 65 78 15 78 32 47

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  2
65 removed

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  3
Queue (front to rear): 78 15 78 32 47

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  ↑↓ for history. Search history with c-↑/c-↓
```

EXPERIMENT 9 - PART 3: Circular Queue Linked List (Enqueue 98, 62, 26, 64 | Dequeue 98 & Display)



```
1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice:  3

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  98
98 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  62
62 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  26
26 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  1
Enter value:  64
64 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  3
Circular Queue: 98 62 26 64 (back to front)

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  2
98 removed

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  3
Circular Queue: 62 26 64 (back to front)

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back:  ↑↓ for history. Search history with c-↑/c-↓
```

Figure: Execution output of Stack, Queue & Circular Queue using Linked List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice: 1

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Enter value: 12
12 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Enter value: 34
34 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Enter value: 42
42 pushed

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 4
Stack (top to bottom): 42 34 12

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 2
42 popped

[Stack] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 5

1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice: 2

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 15
15 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 78
78 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 32
32 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 47
47 inserted

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Queue (front to rear): 65 78 15 78 32 47

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 2
65 removed

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Queue (front to rear): 78 15 78 32 47

[Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 4
```

```
1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice: 3

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 98
98 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 62
62 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 26
26 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Enter value: 64
64 inserted

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Circular Queue: 98 62 26 64 (back to front)

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 2
98 removed

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
Circular Queue: 62 26 64 (back to front)

[Circular Queue] 1.Enqueue 2.Dequeue 3.Display 4.Back: 4

1.Stack  2.Queue  3.Circular Queue  4.Exit
Enter choice: 4
Program ended.
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 10: LINKED LIST CONCATENATION AND POLYNOMIAL REPRESENTATION & ADDITION

AIM: To write and execute a Python program to perform concatenation of two singly linked lists and polynomial representation and addition using linked lists.

Python Source Code

```
# 1. Concatenation of two linked lists
# 2. Polynomial representation and addition using linked lists

# ---------------- PART 1: CONCATENATION ----------------
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def create_list(values):
    head = None
    for v in values:
        node = Node(v)
        if head is None:
            head = node
        else:
            temp = head
            while temp.next:
                temp = temp.next
            temp.next = node
    return head

def concatenate(list1, list2):
    if list1 is None:
        return list2
    temp = list1
    while temp.next:
        temp = temp.next
    temp.next = list2                       # last node of list1 -> head of list2
    return list1

def display(head):
    if head is None:
        print("List is empty")
        return
    while head:
        print(head.data, end=" -> ")
        head = head.next
    print("None")

# ---------------- PART 2: POLYNOMIAL ----------------
class Term:
    def __init__(self, coeff, power):
        self.coeff = coeff
        self.power = power
        self.next = None

def create_poly():
```

```
    head = None
    n = int(input("Enter number of terms: "))
    for i in range(n):
        c = int(input("  Coefficient of term %d: " % (i + 1)))
        p = int(input("  Power of term %d      : " % (i + 1)))
        node = Term(c, p)
        if head is None or p > head.power:  # keep powers in descending order
            node.next = head
            head = node
        else:
            temp = head
            while temp.next and temp.next.power >= p:
                temp = temp.next
            node.next = temp.next
            temp.next = node
    return head

def add_poly(p1, p2):
    result = []
    while p1 and p2:
        if p1.power == p2.power:
            result.append((p1.coeff + p2.coeff, p1.power))
            p1, p2 = p1.next, p2.next
        elif p1.power > p2.power:
            result.append((p1.coeff, p1.power))
            p1 = p1.next
        else:
            result.append((p2.coeff, p2.power))
            p2 = p2.next
    while p1:
        result.append((p1.coeff, p1.power))
        p1 = p1.next
    while p2:
        result.append((p2.coeff, p2.power))
        p2 = p2.next
    head = None
    for c, p in reversed(result):           # build linked list of the sum
        node = Term(c, p)
        node.next = head
        head = node
    return head

def show_poly(head):
    if head is None:
        print("0")
        return
    text = ""
    while head:
        text += "%+dx^%d " % (head.coeff, head.power)
        head = head.next
    print(text)

# ---------------- MAIN MENU ----------------
while True:
    print("\n1. Concatenate two linked lists")
    print("2. Polynomial addition")
    print("3. Exit")
```

```
    ch = input("Enter choice: ")

    if ch == "1":
        a = create_list(input("Enter elements of list 1 (space separated): ").split())
        b = create_list(input("Enter elements of list 2 (space separated): ").split())
        print("List 1 : ", end=""); display(a)
        print("List 2 : ", end=""); display(b)
        print("After concatenation: ", end=""); display(concatenate(a, b))

    elif ch == "2":
        print("\n-- First polynomial --")
        p1 = create_poly()
        print("\n-- Second polynomial --")
        p2 = create_poly()
        print("\nFirst  polynomial : ", end=""); show_poly(p1)
        print("Second polynomial : ", end=""); show_poly(p2)
        print("Sum               : ", end=""); show_poly(add_poly(p1, p2))

    elif ch == "3":
        print("Program ended.")
        break
    else:
        print("Invalid choice.")
```

Space for Output

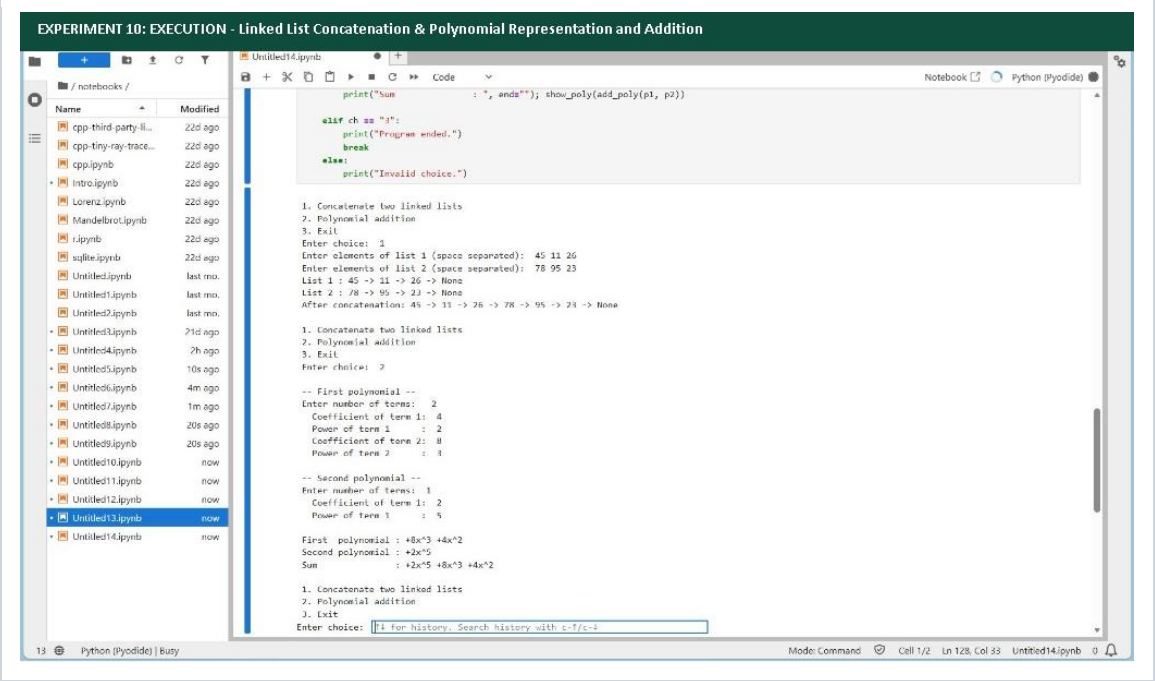


Figure: Execution output of Linked List Concatenation & Polynomial Addition

Sample Run: Execution Log / Console Output (Terminal Execution):

```
1. Concatenate two linked lists
2. Polynomial addition
```

```
3. Exit
Enter choice: 1
Enter elements of list 1 (space separated): 45 11 26
Enter elements of list 2 (space separated): 78 95 23
List 1: 45 -> 11 -> 26 -> None
List 2: 78 -> 95 -> 23 -> None
After concatenation: 45 -> 11 -> 26 -> 78 -> 95 -> 23 -> None

1. Concatenate two linked lists
2. Polynomial addition
3. Exit
Enter choice: 2

-- First polynomial --
Enter number of terms: 2
  Coefficient of term 1: 4
  Power of term 1: 2
  Coefficient of term 2: 8
  Power of term 2: 3

-- Second polynomial --
Enter number of terms: 1
  Coefficient of term 1: 2
  Power of term 1: 5

First  polynomial: +8x^3 +4x^2
Second polynomial: +2x^5
Sum: +2x^5 +8x^3 +4x^2

1. Concatenate two linked lists
2. Polynomial addition
3. Exit
Enter choice: 3
Program ended.
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.